Etapa 5: Separación de Persistencia y Transacciones

Para garantizar que nuestro modelo de dominio (Domain Model) se mantenga agnóstico a la infraestructura y bases de datos, se aplicó el patrón *Repository*. Las interfaces definidas a continuación hablan un lenguaje puro de negocio, ocultando cualquier detalle de consultas SQL o frameworks de persistencia. Estas interfaces interactúan directamente con las entidades principales (*Aggregates*) definidas en el Diagrama de Clases de la Etapa 3.

Diseño de Repositorios:

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
IClienteRepository	Gestionar datos, líneas de crédito y validación de saldos de los clientes.	ObtenerPorCodigo(codigo), ActualizarSaldoCliente(cliente)	Cliente, ClienteEmpresarial.
IInventarioRepository	Controlar niveles de stock, validar mínimos y gestionar traslados de mercancía.	ObtenerExistencia(sku, sucursal), ActualizarStock(existencia)	Existencia, Transferencia.
IVentaRepository	Registrar transacciones exitosas, sus partidas y el comprobante generado.	GuardarVenta(venta), ObtenerPorFolio(folio)	Venta, DetalleVenta, Ticket.
ICotizacionRepository	Administrar folios, fechas de emisión y estados de cotizaciones.	ObtenerPorFolio(folio), GuardarCotizacion(cotizacion)	Cotizacion.

Definición del Alcance Transaccional (Unit of Work)

Para asegurar la consistencia del sistema, implementamos el patrón Unit of Work.

- Inicio: La transacción inicia en la capa de aplicación en el momento exacto en que se invoca el caso de uso.
- Agrupacion de cambios: El Unit of Work rastrea en memoria todas las modificaciones coordinadas. Por ejemplo, al concretar una Venta, agrupa la

creación de la venta, la deducción en la Existencia y la actualización del ClienteEmpresarial.

- Manejo de errores: Si el dominio lanza una excepción por romper una invariante, el Unit of Work aborta la operación y no hace el commit a la base de datos, garantizado que el inventario y el saldo se mantenga intactos.